Dependency Injection

Memisahkan Konstruksi Objek dari Logika Bisnis: Kontras Constructor Injection vs Service Locator di Go

```
go test -race ./... [PASS: 6 Test Cases]
```

Masalah: Hard-Coded Infrastructure di Domain Logic

Tight Coupling Hazard

Ketika sebuah fungsi bisnis menginstansiasi dependensi konkret langsung di dalam method-nya:

- Logika bisnis terkunci pada satu vendor payment atau database driver.
- Mengubah transport protocol memaksa rewrite pada domain layer.
- Konstruksi objek dan eksekusi runtime bercampur aduk.

Dampak Pada Pengujian

Unit test terpaksa berubah menjadi integration test yang lambat dan rapuh:

- Wajib membuka port jaringan atau koneksi database nyata.
- Sulit menguji skenario kegagalan langka secara deterministik.
- Test suites menjadi flaky dan menghambat pipeline CI/CD.

Konteks 5W+1H: What, Why, Who

WHAT & WHY

What: Masalah penguncian implementasi konkret di dalam kelas bisnis dan pemisahan concern konfigurasi dari penggunaan.

Why: Tanpa abstraksi kontrak, setiap refactoring infrastruktur merusak logika bisnis dan menghancurkan testability terisolasi.

WHO

Consumer (Processor): Domain logic yang memproses pembayaran.

Service (PaymentGateway): Abstraksi kontrak infrastruktur.

Injector (main.go): Composition Root yang merakit komponen.

Konteks 5W+1H: When, Where, How

WHEN & WHERE

When: Objek dirakit saat bootstrapping aplikasi (startup phase), sebelum request runtime pertama masuk.

Where: Batas antara Composition Root (`main.go`) dan domain core (`internal/di`).

HOW

Mekanisme: Constructor injection menyalurkan implementasi interface `PaymentGateway` ke dalam `NewProcessor`.

Verifikasi: 6 test assertions memastikan isolasi mock, error propagation, dan input validation guards.

Core Mental Model

"Separate the construction of an object graph from its runtime use. Externalize object creation."

Business modules should never hunt for or build their own dependencies. An external assembler stitches together the object graph at the boundary, allowing domain logic to execute pure behavioral invariants.

Inversion of Control vs Dependency Injection

Inversion of Control (IoC)

Prinsip umum di mana aliran kontrol dialihkan ke external runtime:

- Hollywood Principle: *"Don't call us, we'll call you."*
- Contoh: Web framework dispatchers, event listeners, UI main loops, template methods.

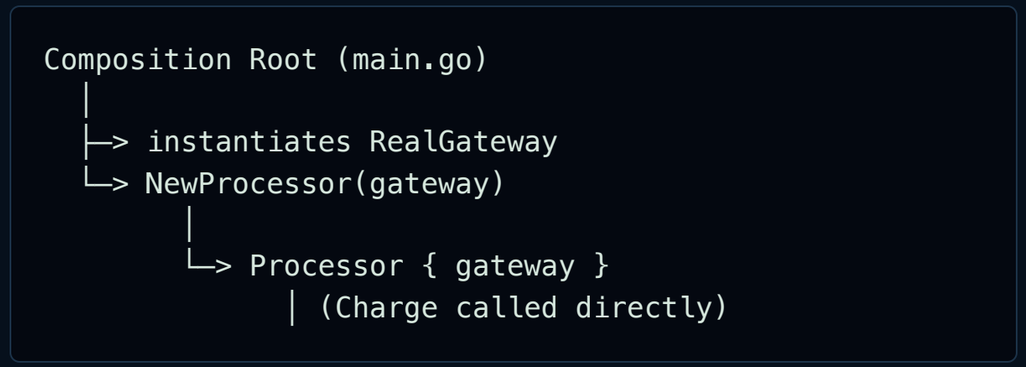
Dependency Injection (DI)

Bentuk spesifik IoC yang difokuskan khusus pada perakitan dependensi:

- Menginversi kontrol penciptaan dependensi ke assembler eksternal.
- Objek menerima dependensi yang dibutuhkan secara pasif.
- Memisahkan lifecycle konstruksi dari eksekusi logika.

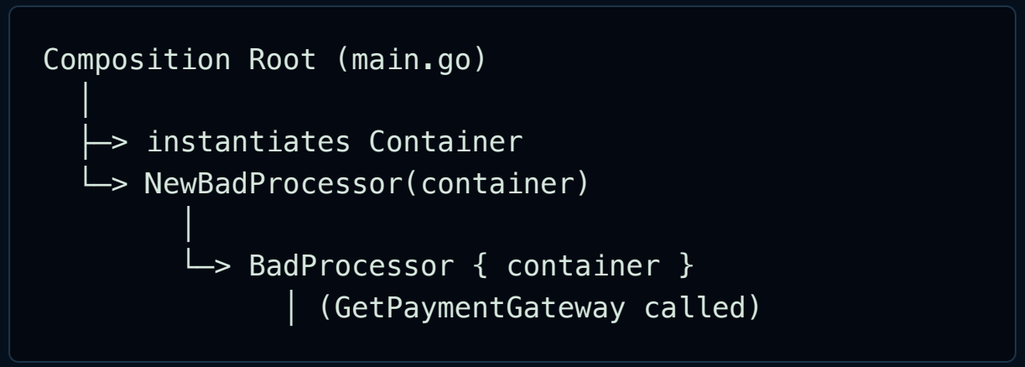
Arsitektur: Constructor Injection vs Service Locator

Jalur Bersih: Constructor Injection



Kontrak eksplisit, valid at birth, tanpa overhead container.

Anti-Pattern: Service Locator



Dependensi tersembunyi, query dinamis di dalam method.

Daftar Komponen & Tanggung Jawab

Komponen	File Lokasi	Peran & Tanggung Jawab
PaymentGateway	internal/di/gateway.go	Interface abstraksi tunggal: `Charge(Money) error`
RealGateway	internal/di/gateway.go	Implementasi konkret mensimulasikan panggilan gateway riil
Money	internal/di/gateway.go	Value object murni (amount & currency) tanpa behavior eksternal
Processor	internal/di/processor.go	Service bisnis menggunakan Constructor Injection eksplisit
BadProcessor	internal/di/locator.go	Service bisnis memegang Service Locator anti-pattern
MockGateway	tests/processor_test.go	Test double penyimpan state panggilan dan kontrol error

Interface Kontrak & Value Object

```go
// internal/di/gateway.go:6-14
type Money struct {
    Amount   int
    Currency string
}

type PaymentGateway interface {
    Charge(m Money) error
}
```

Desain Value Object

Money adalah data murni tanpa side effects:

- Tidak memerlukan interface abstraksi.
- Diinstansiasi langsung di call-site: `Money{Amount: 50, Currency: "USD"}`.
- Memasukkan value object ke dalam container DI adalah anti-pattern yang menambah kompleksitas tanpa manfaat.

Implementasi: Constructor Injection

```go
// internal/di/processor.go:6-21
type Processor struct {
    gateway PaymentGateway
}

func NewProcessor(g PaymentGateway) *Processor {
    return &Processor{gateway: g}
}

func (p *Processor) ProcessPayment(amount int) error {
    if amount <= 0 {
        return errors.New("invalid amount")
    }
    m := Money{Amount: amount, Currency: "USD"}
    return p.gateway.Charge(m)
}
```

Keunggulan Arsitektur

- **Valid at Birth:** Objek dijamin lengkap sejak dibuat; tidak ada uninitialized pointer.
- **Immutability:** Dependensi disimpan di unexported field, aman dari mutasi liar.
- **Zero Container Coupling:** Struct tidak mengenal container, framework, atau reflection.

Implementasi: Service Locator Anti-Pattern

```go
// internal/di/locator.go:6-25
type Container interface {
    GetPaymentGateway() PaymentGateway
}

type BadProcessor struct {
    container Container
}

func NewBadProcessor(c Container) *BadProcessor {
    return &BadProcessor{container: c}
}

func (p *BadProcessor) ProcessPayment(amount int) error {
    if amount <= 0 { return errors.New("invalid amount") }
    m := Money{Amount: amount, Currency: "USD"}
    return p.container.GetPaymentGateway().Charge(m)
}
```

Biaya Tersembunyi

- **Hidden Dependencies:** Constructor menyamarkan service yang sebenarnya dipakai.
- **Framework Coupling:** Kelas bisnis bergantung pada API container locator.
- **PSR-11 Warning:** Standard modern menyatakan objek domain TIDAK BOLEH menerima container.

Composition Root: cmd/demo/main.go

```go
// cmd/demo/main.go:16-35
func main() {
    realGateway := &di.RealGateway{}

    // 1. Constructor Injection
    processor := di.NewProcessor(realGateway)
    fmt.Println("--- Running Constructor Injection ---")
    _ = processor.ProcessPayment(100)

    // 2. Service Locator (Anti-Pattern)
    container := &SimpleContainer{gateway: realGateway}
    badProcessor := di.NewBadProcessor(container)
    fmt.Println("--- Running Service Locator ---")
    _ = badProcessor.ProcessPayment(200)
}
```

Peran Composition Root

Titik sentral tempat seluruh komponen dihubungkan:

- Satu-satunya lokasi yang membuat instance konkret `RealGateway`.
- Domain layer sama sekali tidak tahu file konfigurasi atau database credentials.
- Mengubah implementasi hanya membutuhkan pengubahan satu baris di `main.go`.

Eksekusi Runtime Demo

```
$ go run ./cmd/demo
--- Running Constructor Injection ---
RealGateway charging 100 USD
--- Running Service Locator ---
RealGateway charging 200 USD
```

Kompilasi dan eksekusi membuktikan kedua pola menghasilkan output fungsional yang sama saat runtime.

Observasi Runtime

Meskipun output runtime serupa:

- Constructor Injection menyelesaikan dependensi saat compile-time / assembly.
- Service Locator menambah layer lookup dinamis setiap kali method dijalankan.
- Kemudahan membaca dan menelusuri alur kode jauh lebih unggul pada Constructor Injection.

Failure Scenario 1: Input Validation Pre-Check

```go
// Pre-check sebelum panggil gateway
if amount <= 0 {
    return errors.New("invalid amount")
}
m := Money{Amount: amount, Currency: "USD"}
return p.gateway.Charge(m)
```

Pelindung Infrastruktur

Validasi domain menghentikan eksekusi sebelum menyentuh infrastruktur:

- Mencegah request jaringan yang tidak valid dan membuang resource.
- DI tidak menggantikan input validation—keduanya saling melengkapi.
- Test membuktikan `mock.ChargedMoney.Amount` tetap 0 saat amount negatif.

Failure Scenario 2: Error Propagation

```go
// Mock simulasi gateway error
func (m *MockGateway) Charge(money di.Money) error {
    if m.ShouldFail {
        return errors.New("gateway unavailable")
    }
    m.ChargedMoney = money
    return nil
}
```

Propagasi Tanpa Distorsi

Ketika gateway downstream gagal:

- Error dikembalikan langsung ke pemanggil tanpa ditelan atau diabaikan.
- Caller dapat menentukan strategi mitigasi: retry, fallback, atau circuit break.
- Pengujian membuktikan error diteruskan secara utuh pada kedua processor.

Edge Cases: Nil Injection & Immutability

Bahaya Nil Pointer di Go

Jika caller mengirim `nil` ke `NewProcessor(nil)`:

- Kompilasi tetap sukses karena `nil` adalah nilai interface yang sah.
- Pemanggilan `p.gateway.Charge(m)` akan memicu `panic: runtime error`.
- Rekomendasi produksi: Tambahkan nil-check eksplisit di constructor jika batas kepercayaan longgar.

Immutability Dependensi

Field `gateway` bersifat unexported (huruf kecil):

- Tidak ada method setter yang memungkinkan penggantian dependensi secara acak di tengah jalan.
- Mencegah race condition mutasi dependensi saat concurrent execution.
- Konsisten dengan prinsip objek stabil sepanjang masa pakainya.

Test Suite: 6 Skenario Terisolasi

Test Function	Target Komponen	Kondisi Pengujian & Assertion
`TestProcessor_Success`	`Processor` (DI)	Amount 50: error nil, amount 50 & currency USD tercatat
`TestProcessor_GatewayError`	`Processor` (DI)	`ShouldFail: true`: error diteruskan ke caller
`TestProcessor_InvalidAmount`	`Processor` (DI)	Amount -10: return error, `ChargedMoney.Amount == 0`
`TestBadProcessor_Success`	`BadProcessor` (Locator)	Amount 75: error nil, amount 75 tercatat via mock container
`TestBadProcessor_GatewayError`	`BadProcessor` (Locator)	`ShouldFail: true`: error diteruskan via mock container
`TestBadProcessor_InvalidAmount`	`BadProcessor` (Locator)	Amount -5: return error, `ChargedMoney.Amount == 0`

Test Double: MockGateway & MockContainer

```go
// tests/processor_test.go:10-21, 61-67
type MockGateway struct {
    ChargedMoney di.Money
    ShouldFail   bool
}

type MockContainer struct {
    gateway di.PaymentGateway
}

func (c *MockContainer) GetPaymentGateway() di.PaymentGateway {
    return c.gateway
}
```

Kontras Setup Pengujian

Perbedaan usaha pengujian terlihat jelas:

- **Processor (DI):** Cukup mengoper `mock` langsung ke `NewProcessor(mock)`.
- **BadProcessor (Locator):** Wajib membuat wrapper `MockContainer` terlebih dahulu sebelum bisa mengetes processor.
- Service Locator menambah kompleksitas setup pengujian tanpa nilai tambah.

Eksekusi Pengujian: Race Safety Terbukti

```
$ go test -race ./...
?   lab16/cmd/demo  [no test files]
?   lab16/internal/di  [no test files]
ok  lab16/tests     1.442s
```

Pengujian dieksekusi dengan race detector aktif tanpa mendeteksi data race apapun.

Fakta Hasil Eksekusi

- 6/6 test assertions lulus 100%.
- Eksekusi unit test instan (0.00s execution per test).
- Zero dependency: tidak membuka socket jaringan, port HTTP, atau koneksi DB.

Batas Pembuktian: Apa yang Belum Dibuktikan

Terbukti Secara Nyata

- Pemisahan konfigurasi dari penggunaan di `main.go`.
- Isolasi test 100% menggunakan mock double.
- Kontras struktural Constructor vs Service Locator.
- Penolakan input invalid sebelum pemanggilan gateway.

Batas & Omission Lab

- Tidak ada DI Framework otomatis (Google Wire, Uber Dig).
- Lifecycle management (Singleton, Scoped, Transient) tidak diuji.
- Overhead performa atau alokasi memori tidak diukur.
- Tidak ada bukti statistik empiris terkait penurunan bug di industri.

Trade-offs: Kapan Menggunakan DI

Kapan Tepat Digunakan

- Komponen berinteraksi dengan infrastruktur eksternal (DB, HTTP, queue).
- Komponen membutuhkan pergantian implementasi atau mocking saat testing.
- Aplikasi memiliki arsitektur modular dengan banyak bounded context.

Biaya & Kompleksitas Tambahan

- Menambah boilerplate kode perakitan di Composition Root.
- Menyulitkan penelusuran kode jika menggunakan framework reflection.
- Hindari over-engineering pada value objects murni atau utilitas internal sederhana.

Pelajaran Inti Software Engineering

1. Constructor Injection Adalah Default

Gunakan konstruktor eksplisit untuk memastikan setiap objek valid sejak inisialisasi dan dependensi terlihat transparan.

2. Hindari Service Locator di Domain

Service Locator menyembunyikan dependensi, mengikat kode domain pada framework, dan memperumit unit testing.

3. Jangan Masukkan Value Object ke DI

Struktur data murni seperti `Money` cukup diinstansiasi langsung tanpa perantara interface atau container.

4. Constructor Bloat = Refactoring Signal

Banyaknya parameter di constructor adalah indikator pelanggaran SRP, bukan alasan beralih ke Service Locator.

Production Readiness Checklist

Desain & Arsitektur

- [ ] Dependensi dideklarasikan sebagai interface minimal.
- [ ] Constructor injection digunakan secara konsisten.
- [ ] Tidak ada dependensi ke container di dalam logika domain.
- [ ] Composition Root terisolasi di entry point (misal: `main.go`).

Pengujian & Validasi

- [ ] Seluruh domain unit test berjalan terisolasi memakai test double.
- [ ] Validasi input pre-check dijalankan sebelum memanggil dependensi.
- [ ] Error propagation dari downstream diverifikasi secara akurat.
- [ ] `go test -race ./...` lulus tanpa race condition.

Software Engineering Lab 16

Pemisahan konstruksi dari penggunaan menghasilkan arsitektur yang modular, mudah diuji, dan adaptif terhadap perubahan infrastruktur.

GitHub Repository:
https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection

Commit Hash: 8afe2a824667528d16d6d9b2ac434a18a6ac2db1 · Go 1.22